Day 113 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 17, Day 2 reference

Watch: Support Vector Machines Part 2: The Polynomial Kernel (StatQuest) | Support Vector Machines Part 3: The Radial (RBF) Kernel (StatQuest)

`day113_kernel_trick.py`, segment by segment, in the order `main()` uses things. Day 112's SMO is imported unchanged; the kernels are new. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: K = kernel matrix · ϕ = the hidden feature map · γ = RBF width · D = random features · W, c = random frequencies and phases

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day112_support_vector_machines as d112
smo, bias, standardise, stratified_split = d112.smo, d112.bias, d112.standardise,
    d112.stratified_split
d99 = d112.d99
GAMMAS = [0.01, 0.1, 1, 10, 100]
GRID_G = [0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1, 3]
GRID_C = [0.1, 1, 10, 100]
DIMS = [10, 30, 100, 300, 1000, 3000]
SPLITS = 5
```

- `smo, bias, standardise, stratified_split = d112.smo, ...` — Day 112's functions under short names, unchanged.
- `GRID_G = [0.001, ..., 1, 3]` — 1 and 3 were added after the first run chose 0.3, the old edge of the grid.

Segment 1 — Kernels and features

The kernels

MATH

$$|x - z|^2 = |x|^2 + |z|^2 - 2x{\cdot}z; \quad K = exp(-\gamma|x - z|^2)$$

CODE

```
def linear(A, B):
    return A @ B.T
def poly(A, B, degree=2, c=1.0):
    return (A @ B.T + c) ** degree
def sqdist(A, B):
    return (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
def rbf(A, B, gamma):
    return np.exp(-gamma * np.maximum(sqdist(A, B), 0))
```

- `(A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T` — |a - b|^2 = |a|^2 + |b|^2 - 2 a.b, for every pair, by broadcasting a column against a row.
- `np.maximum(..., 0)` — rounding can make a zero distance slightly negative; clip it.
- `(A @ B.T + c) ** degree` — ** on an array raises every entry to the power.

phi_poly2 and rff: explicit features

MATH

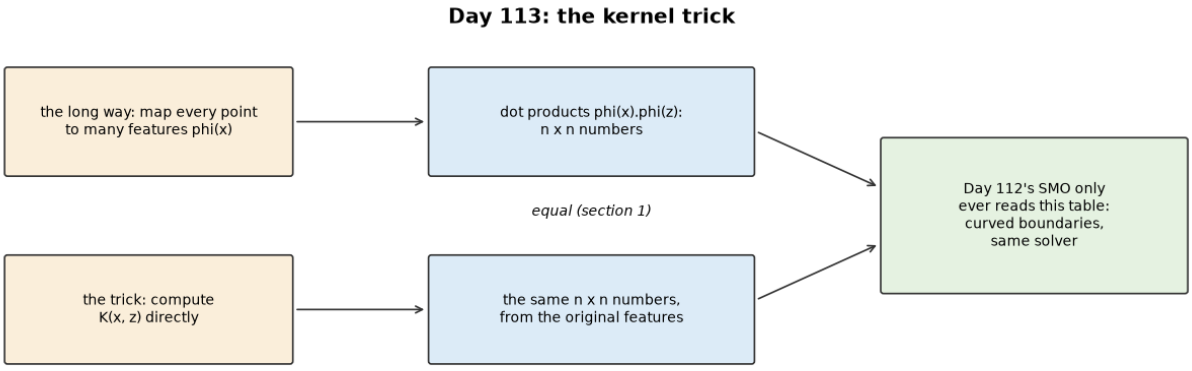
$$z(x) = \sqrt{(2/D)}\ cos(Wx + c), \quad W \sim Normal(0, 2\gamma),\ c \sim Uniform(0, 2\pi)$$

CODE

```
def phi_poly2(X):
    n, p = X.shape; cols = [np.ones(n)] + [np.sqrt(2) * X[:, i] for i in range(p)]
    cols += [X[:, i] ** 2 for i in range(p)]
    cols += [np.sqrt(2) * X[:, i] * X[:, j] for i in range(p) for j in range(i + 1, p)]
    return np.column_stack(cols)
def rff(X, D, gamma, seed):
    rng = np.random.RandomState(seed)
    W = rng.normal(scale=np.sqrt(2 * gamma), size=(X.shape[1], D))
    c = rng.uniform(0, 2 * np.pi, D)
    return lambda Z: np.sqrt(2.0 / D) * np.cos(Z @ W + c)
```

- `[... for i in range(p) for j in range(i + 1, p)]` — a double loop in one list: every pair once.
- `np.column_stack(cols)` — joins the columns into one table.
- `return lambda Z: ...` — a function that remembers W and c, so training and test rows get the same map.

SYSTEM DIAGRAM



The kernel trick.

Segment 2 — A kernel SVM

fit_kernel_svm and one-vs-rest

CODE

```
def fit_kernel_svm(K, y, C):
    a, it = smo(K, y, C)
    return a, bias(a, y, K, C), it
def ovr_fit(K, y, C):
    return [fit_kernel_svm(K, np.where(y == c, 1.0, -1.0), C)[:2] for c in range(int(y.max()) +
        1)]
def ovr_predict(models, y, K_new):
    scores = [K_new.T @ (a * np.where(y == c, 1.0, -1.0)) + b for c, (a, b) in enumerate(models)]
    return np.argmax(np.stack(scores, 1), 1)
```

- `np.where(y == c, 1.0, -1.0)` — turns ten labels into +1 for one digit and -1 for the rest.
- `K_new.T @ (a * np.where(...)) + b` — each SVM's score for every new point.
- `np.stack(scores, 1)` — one column per class; argmax along it picks the most confident.

Segment 3 — The experiments

Eigenvalues, inside demo_valid

CODE

```
e = np.linalg.eigvalsh((K + K.T) / 2)
```

- `np.linalg.eigvalsh` — eigenvalues of a symmetric matrix, in increasing order.
- `(K + K.T) / 2` — removes any tiny asymmetry from rounding before the test.

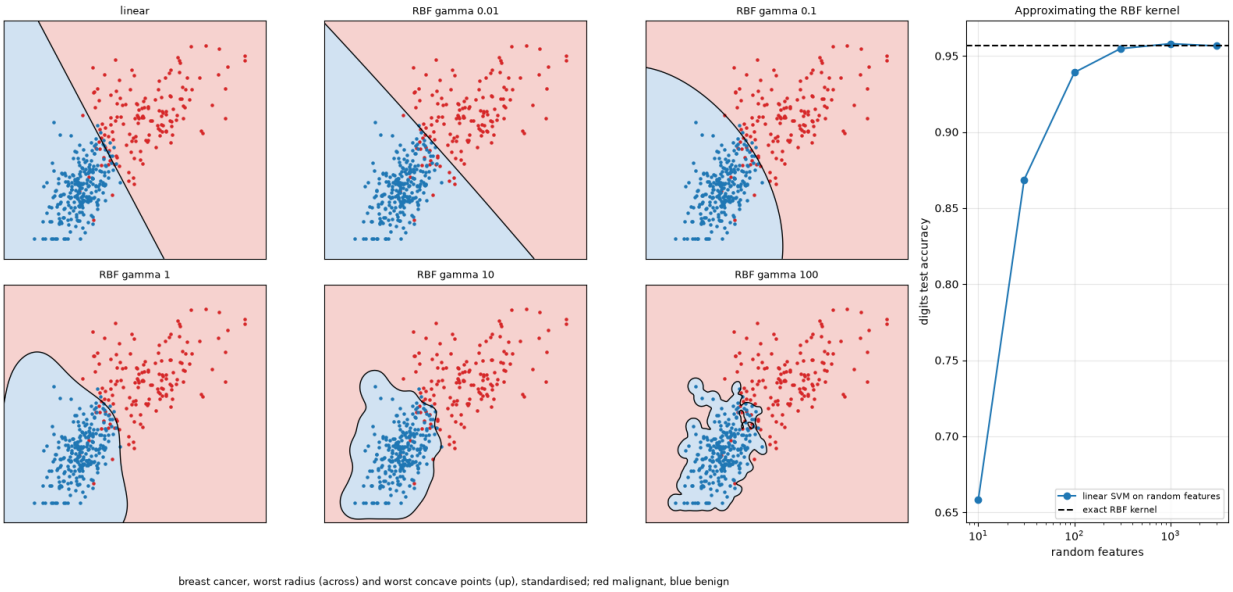
The grid, inside demo_grid

CODE

```
S = np.zeros((len(GRID_G), len(GRID_C))); SV = np.zeros_like(S)
for i, g in enumerate(GRID_G):
    for j, C in enumerate(GRID_C):
        acc = []
        for k in range(3):
            va = folds[k]; ft = np.concatenate([folds[m] for m in range(3) if m != k])
            m = ovr_fit(rbf(X[ft], X[ft], g), y[ft], C); acc.append(np.mean(ovr_predict(m, y[ft],
                rbf(X[ft], X[va], g)) == y[va]))
            SV[i, j] += np.mean([np.mean(a > 1e-8) for a, _ in m]) / 3
        S[i, j] = np.mean(acc)
print(f"{'CV accuracy':<14}" + "".join(f"{'C ' + str(c):>10}" for c in GRID_C))
for i, g in enumerate(GRID_G):
    print(f"{'gamma ' + str(g):<14}" + "".join(f"{S[i, j]:>10.4f}" for j in range(len(GRID_C))))
print(f"{'support share':<14}" + "".join(f"{'C ' + str(c):>10}" for c in GRID_C))
for i, g in enumerate(GRID_G):
    print(f"{'gamma ' + str(g):<14}" + "".join(f"{SV[i, j]:>10.3f}" for j in range(len(GRID_C))))
i, j = np.unravel_index(np.argmax(S), S.shape); g, C = GRID_G[i], GRID_C[j]
```

- `np.unravel_index(np.argmax(S), S.shape)` — argmax counts through the table as one long list; this turns that position back into (row, column).
- `SV[i, j] += ... / 3` — the share of support vectors, averaged over the three folds.

REAL GRAPH



Real graph from the script: boundaries for each kernel, and random features against the exact kernel.

Quick Syntax Glossary — new or reinforced constructs

- `a[:, None] + b[None, :]` — broadcasting: every a against every b.
- `np.linalg.eigvalsh(M)` — eigenvalues of a symmetric matrix.
- `np.unravel_index(i, shape)` — a flat position to (row, column).
- `lambda Z: f(Z, W, c)` — a function that keeps W and c from where it was made.
- `from math import comb` — comb(n, k): the number of ways to choose k of n.
- `np.column_stack(list)` — columns side by side.

End of Day 113 syntax reference. For the experiments and what they showed, see the Day 113 complete guide.